Summa

Reference manual

Every function, option and convention, with an example computed by the package

Nazzareno Africani · SISSA · version 1.0.0 · November 2026

Contents

1 Installation and loading 1

2 Conventions 2

3 Einsum 4

4 EinsumPath 5

5 EinsumFunction 6

6 EinsumDiagram 9

7 EinsumTree 12

8 The plan cache 13

9 Messages 14

10 Performance 16

11 Limitations 16

12 Citing Summa 17

13 License 17

1 Installation and loading

Summa is a paclet. Install the paclet of the latest release once, then load it in every session; the guided tour comes with the paclet:

```
PacletInstall["https://github.com/n4sd4q/Summa/releases/download/v1.0.0/Summa-1.0.0.paclet"]
Needs["Summa`"]
NotebookOpen[PacletObject["Summa"]["AssetLocation", "Examples"]]
```

To use a clone of the repository without installing, load its folder before `Needs`:

```
PacletDirectoryLoad["/path/to/Summa/Summa"]
```

Summa has no dependencies. It needs Mathematica 12.1 or later and is tested with 15.0. In the examples of this manual it is loaded by

```
Needs["Summa`"]
```

`$SummaVersion`

The version of the package, a string.

```
{$SummaVersion, $SummaUpdateCheck}
```

```
{"1.0.0", True}
```

Loaded in a notebook, the package prints a banner: its logo, the attribution "Summa, by Nazzareno Africani", the version, the license, how to cite it, and links to the repository and the website. A script prints nothing and never checks for updates.

`SummaUpdateCheck, $SummaUpdateCheck`

At most once a day, in the background, loading the package in a notebook asks GitHub (`api.github.com`) for the latest release and, if it is newer than `$SummaVersion`, adds to the banner the `PacletInstall` line of that release; any failure is silent, and a session without access to the Internet never asks. `$SummaUpdateCheck = False` before `Needs["Summa`"]` (in `init.m` for good) turns the check off; the setting is kept when the package is loaded again. `SummaUpdateCheck[]` asks by hand and waits for the answer: it gives `<|"InstalledVersion" -> ..., "LatestVersion" -> ..., "InstallLine" -> ...|>`, and prints the `PacletInstall` line when the latest release is newer; when the release cannot be read it gives `$Failed` with the message `SummaUpdateCheck::latest`. This is the only access of the package to the network.

The test suite runs with `wolfram -script Tests/RunTests.wls`.

2 Conventions

- **Subscripts.** In a string specification each index is one character: letters, digits or any Unicode letter, case-sensitive (`i` and `I` are different indices). Subscripts are separated by commas; spaces, tabs and new lines are ignored. The arrow is `->`, → (`\[RightArrow]`), `\[LongRightArrow]` or `\[Rule]`. In the list syntax a subscript is a list, and an index is any expression.
- **Summation.** An index that appears in the operands and not in the output is summed over; an index of the output is kept. An index can appear any number of times, within one operand (diagonals and traces) and across operands (batch indices and hyperedges). An index of the output cannot repeat, as in numpy.
- **Implicit output.** Without the arrow, the output has the indices that appear exactly once, sorted: by character code in a string specification (upper case before lower case), in canonical order (`Sort`) in the list syntax; the indices of the ellipsis come first.
- **Ellipsis.** `...` (or `…`) stands for the leading axes that a subscript does not name, at most once per subscript. The ellipses of the operands are aligned from the right and broadcast as in numpy; when the operands have such axes, an explicit output must contain `...`.
- **Dimensions.** An index has one dimension in all its places; an axis of size 1 is broadcast against a larger one. The places of an index repeated within one operand must have the same dimension.
- **Operands.** A tensor is a rectangular list (packed or not) or a `SparseArray`; any other argument, `Sqrt[2]` or `a + b` included, is a scalar, with the empty subscript. Entries can be

numbers of any kind or symbolic expressions. A result with no index is a scalar.

- **Paths.** A contraction path is a list of pairs in the linear format of numpy and opt_einsum, 1-based: each pair gives the positions of two tensors in the current list, which are removed; their result is appended at the end.
- **Costs.** The cost of a pairwise contraction is the number of terms of its sum, the product of the dimensions of all its indices; the cost of a path (`"Flops"`) is the sum over its steps, plus the size of each operand with a diagonal or an index summed alone. The naive cost (`"NaiveFlops"`) is the product of the dimensions of all the indices times the number of operands minus one (at least one).

Spaces, Unicode letters and the arrow →; and the case of the indices:

```
m = {{1, 2}, {3, 4}}; n = {{0, 1}, {1, 1}};
{Einsum["ij,jk->ik", m, n], Einsum[" i j , j k -> i k ", m, n],
 Einsum["αβ,βγ→αγ", m, n]}
```

```
{{{2, 3}, {4, 7}}, {{2, 3}, {4, 7}}, {{2, 3}, {4, 7}}}
```

```
Einsum["iI,iI->", m, n] == Total[m n, 2]
```

```
True
```

The implicit output, in a string and in the list syntax (`"Spec"` of `EinsumPath` writes it out):

```
{EinsumPath["iB,ia", {{2, 3}, {2, 4}}]["Spec"],
 EinsumPath[{{x, 2}, {2, "y"}}, {{2, 3}, {3, 4}}]["Spec"]}
```

```
{"iB,ia->Ba", "{x,2},{2,y}->{y,x}"}
```

The ellipsis comes first in the implicit output; it can also stand for trailing axes:

```
SeedRandom[1];
s1 = RandomInteger[9, {6, 5, 2, 3}]; s2 = RandomInteger[9, {5, 3, 4}];
{Dimensions[Einsum["...ij,...jk", s1, s2]],
 Dimensions[Einsum["i...,i...->...", m, n]]}
```

```
{{6, 5, 2, 4}, {2}}
```

Broadcasting of an axis of size 1, and the dimensions of a diagonal, which are not broadcast:

```
Einsum["ij,ij->ij", {{1, 2}, {3, 4}, {5, 6}}, {{10, 100}}]
```

```
{{10, 200}, {30, 400}, {50, 600}}
```

```
Einsum["ii->i", {{1, 2, 3}}]
```

```
Einsum::dim: Index i has incompatible dimensions {1, 3}.
$Failed
```

Scalars, and a sum over all the entries:

```
{Einsum[",->", 2, 3], Einsum["ij,->ij", m, x], Einsum["i->", {a, b, c}]}
```

```
{6, {{x, 2*x}, {3*x, 4*x}}, a + b + c}
```

3 Einsum

`Einsum`

`Einsum["spec", t1, t2, ...]` evaluates the einsum of the string spec, as `"ij,jk->ik"`. `Einsum[{sub1, sub2, ...} -> out, t1, t2, ...]` takes the subscripts as lists of indices (or as strings), `Einsum[{sub1, sub2, ...}, ...]` has the implicit output and `Einsum["in" -> "out", ...]` is the string form written as a rule. The order of the contractions is chosen, each pairwise contraction runs on a kernel (BLAS `Dot`, a batched or a broadcasting product), and the plan is cached.

```
{Einsum[{"ij", "jk"} -> "ki", m, n], Einsum["ij" -> "ji", m],
 Einsum[{{1, 2}, {2, 3}} -> {3, 1}, m, n], Einsum[{{"row", "col"}}, m]}
```

```
{{{2, 4}, {3, 7}}, {{1, 3}, {2, 4}}, {{2, 4}, {3, 7}}, {{1, 3}, {2, 4}}}
```

Options:

option	default	values
`"Path"`	`Automatic`	`Automatic`: `"Optimal"` up to 8 tensors, `"Greedy"` beyond; `"Optimal"`: the cheapest path, by dynamic programming over the subsets of the tensors (up to 14 tensors); `"Greedy"`: at each step the pair that least increases the size (opt_einsum); `"Sequential"`: from left to right; `{{i, j}, ...}`: a complete path of your own (linear format, 1-based)
`"BatchMethod"`	`Automatic`	the kernel of a batched contraction: `Automatic` (a cost model), `"Dot"` (a loop of `Dot`) or `"Vectorized"` (products and `Total` on packed arrays)
`"Cache"`	`True`	`False`: the plan is made again and not kept

The result does not depend on the options, only the time to get it:

```
SeedRandom[2]; {ma, mb, mc} = RandomInteger[{-9, 9}, {3, 4, 4}];
Table[Einsum["ij,jk,kl->il", ma, mb, mc, "Path" -> p] == ma . mb . mc,
 {p, {Automatic, "Optimal", "Greedy", "Sequential", {{2, 3}, {1, 2}}}}]
```

```
{True, True, True, True, True}
```

```
SeedRandom[3];
xs = RandomInteger[{-9, 9}, {50, 3, 4}]; ys = RandomInteger[{-9, 9}, {50, 4, 2}];
Table[Einsum["bij,bjk->bik", xs, ys, "BatchMethod" -> b] ==
  MapThread[Dot, {xs, ys}], {b, {Automatic, "Dot", "Vectorized"}}]
```

```
{True, True, True}
```

```
Einsum["ij,jk->ik", m, n, "Cache" -> False]
```

```
{{2, 3}, {4, 7}}
```

An unknown option gives a message and is ignored:

```
Einsum["ij->ji", m, "Foo" -> 1]
```

```
Einsum::optx: Unknown option Foo in Einsum.
{{1, 3}, {2, 4}}
```

4 EinsumPath

`EinsumPath`

`EinsumPath[spec, t1, t2, ...]` or `EinsumPath[spec, {dims1, dims2, ...}]` describes the contraction path of an einsum and its cost, from the shapes alone: an association with

key	
`"Spec"`	the specification, written out (with the implicit output)
`"Path"`	the path, in the linear format of numpy, 1-based
`"Method"`	`"Optimal"`, `"Greedy"`, `"Sequential"`, `"User"`, or `"Trivial"` (one or two tensors)
`"Contractions"`	the specification of each pairwise contraction
`"StepFlops"`	the cost of each pairwise contraction
`"Flops"`	the cost of the path
`"NaiveFlops"`	the cost of the sum written out over all the indices at once
`"Speedup"`	`"NaiveFlops"/"Flops"`
`"LargestIntermediate"`	the size of the largest tensor built on the way

Option `"Path"`, with the values of `Einsum`. The product of three matrices: A times B first, which is cheaper than B times C first:

```
EinsumPath["ij,jk,kl->il", {{2, 30}, {30, 40}, {40, 3}}]
```

```
<|"Spec" -> "ij,jk,kl->il", "Path" -> {{1, 2}, {1, 2}}, "Method" -> "Optimal",
 "Contractions" -> {"ij,jk->ik", "ik,kl->il"}, "StepFlops" -> {2400., 240.},
 "Flops" -> 2640., "NaiveFlops" -> 14400., "Speedup" -> 5.45455,
 "LargestIntermediate" -> 80.|>
```

```
userPath = EinsumPath["ij,jk,kl->il", {{2, 30}, {30, 40}, {40, 3}},
   "Path" -> {{2, 3}, {1, 2}}];
userPath[[{"Path", "Method", "Contractions", "StepFlops", "Flops"}]]
```

```
<|"Path" -> {{2, 3}, {1, 2}}, "Method" -> "User",
 "Contractions" -> {"jk,kl->jl", "ij,jl->il"}, "StepFlops" -> {3600., 180.},
 "Flops" -> 3780.|>
```

The tensors or their dimensions give the same description:

```
EinsumPath["ij,jk,kl->il", RandomReal[1, {2, 30}], RandomReal[1, {30, 40}],
   RandomReal[1, {40, 3}]] ===
 EinsumPath["ij,jk,kl->il", {{2, 30}, {30, 40}, {40, 3}}]
```

```
True
```

With one or two tensors there is nothing to choose; an operand with a diagonal or an index summed alone costs its size, and the speed-up can then be below 1:

```
{EinsumPath["ij,jk->ik", {{2, 3}, {3, 4}}]["Method"],
 EinsumPath["iij->j", {{3, 3, 4}}][[{"Flops", "NaiveFlops"}]]}
```

```
{"Trivial", <|"Flops" -> 36., "NaiveFlops" -> 12.|>}
```

5 EinsumFunction

`EinsumFunction`

`EinsumFunction[spec, {dims1, dims2, ...}]` makes the plan of an einsum once, for tensors of the given dimensions ({} for a scalar), and gives a function: `ef[t1, t2, ...]` runs the plan. Tensors of other dimensions give `Einsum::mismatch` and `$Failed`. Options `"Path"` and `"BatchMethod"`, as `Einsum`. In a notebook it shows a summary box with the icon of the network, the specification and the cost; opened, it shows the shapes, the naive cost, the speed-up, the largest intermediate and the path.

```
ef = EinsumFunction["bij,bjk->bik", {{4, 3, 2}, {4, 2, 5}}]
```

```
EinsumFunction[ Spec: bij,bjk->bik
                Flops: 120 ]
```

```
SeedRandom[4]; bx = RandomInteger[9, {4, 3, 2}]; by = RandomInteger[9, {4, 2, 5}];
ef[bx, by] == Table[bx[[b]] . by[[b]], {b, 4}]
```

```
True
```

```
EinsumFunction[",ij->ij", {{}, {2, 2}}][Sin[x], m]
```

```
{{Sin[x], 2*Sin[x]}, {3*Sin[x], 4*Sin[x]}}
```

Its properties:

property	
"Spec", "Path", "Method", "Flops", ...	"NaiveFlops", "Speedup", "LargestIntermediate", "Contractions": as in EinsumPath
"Shapes"	the dimensions of the operands
"Diagram"	EinsumDiagram of the einsum, with the dimensions and "ShowPath" -> True
"Tree"	EinsumTree of the einsum, with the dimensions
"Plan"	the plan itself, an association, for inspection (its form may change between versions)
"Properties"	the list of the properties

```
ef["Properties"]
```

```
{"Spec", "Shapes", "Path", "Method", "Flops", "NaiveFlops", "Speedup",
 "LargestIntermediate", "Contractions", "Diagram", "Tree", "Plan"}
```

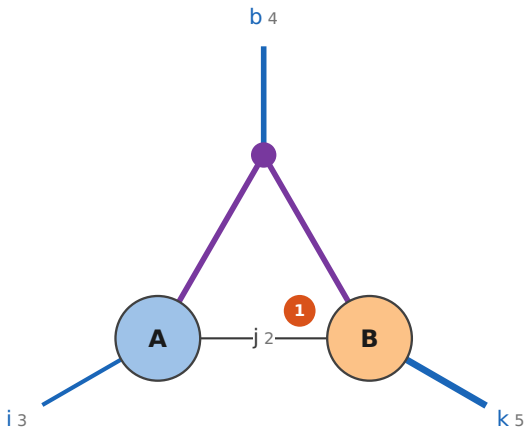
```
AssociationMap[ef, {"Spec", "Shapes", "Path", "Method", "Flops", "NaiveFlops",
  "Speedup", "LargestIntermediate", "Contractions"}]
```

```
<|"Spec" -> "bij,bjk->bik", "Shapes" -> {{4, 3, 2}, {4, 2, 5}},
 "Path" -> {{1, 2}}, "Method" -> "Trivial", "Flops" -> 120.,
 "NaiveFlops" -> 120., "Speedup" -> 1., "LargestIntermediate" -> 60.,
 "Contractions" -> {"bij,bjk->bik"}|>
```

```
Keys[ef["Plan"]]
```

```
{"Spec", "Shapes", "Names", "Inputs", "Output", "Dims", "Type", "Pre", "Sets",
 "SSA", "Method", "Steps", "Final", "FinalPerm", "Flops", "NaiveFlops",
 "LargestIntermediate"}
```

```
ef["Diagram"]
```




```
ef[by, bx]
```

```
Einsum::mismatch: The tensors have dimensions {{4, 2, 5}, {4, 3, 2}}, but the
    EinsumFunction was compiled for {{4, 3, 2}, {4, 2, 5}}.
$Failed
```

6 EinsumDiagram

`EinsumDiagram`

`EinsumDiagram[spec]` draws the tensor network of an einsum; `EinsumDiagram[spec, t1, t2, ...]` or `EinsumDiagram[spec, {dims1, dims2, ...}]` also writes the dimensions. Every index is a COPY tensor with one leg for each time it appears, plus one if it is in the output, and is drawn by the number of its legs:

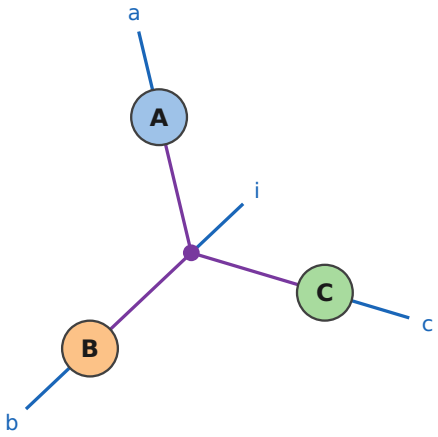
index	drawn as
two legs, in two tensors	a bond, with the name of the index in the middle
two legs, in one tensor and the output	an open leg, in blue
two legs, in one tensor	a loop: a trace
one leg (summed in one tensor alone)	a dashed leg ending in a small circle: a contraction with the vector of ones
three legs or more	a hyperedge: a purple dot joined to the tensors (and an open leg if it is in the output)

Bonds get thicker as their dimension grows (on a logarithmic scale); tensors with the same name have the same colour; in a notebook, a tooltip on each element tells its indices, dimensions and role. The automatic layout is classical multidimensional scaling of the distances in the graph, refined by stress majorization, with the dangling legs in the free angular gaps. Options:

option	default	
`"TensorNames"`	`Automatic`	the names of the symbols passed as tensors, otherwise A, B, C, ...; or a list
`"ShowDimensions"`	`Automatic`	the dimensions next to the indices, when they are known; `False`
`"ShowPath"`	`False`	`True`: each bond marked with the step at which it is contracted
`"Path"`	`Automatic`	the path drawn by `"ShowPath"` and `"Steps"`, as in `Einsum`
`"Layout"`	`Automatic`	`"Circle"`, `"Line"`, or the coordinates of the tensors, `{{x1, y1}, ...}`
`"Highlight"`	`{}`	the positions of the tensors drawn with a thick border
`"Steps"`	`False`	`True`: a list of pictures, one per step and one of the result
`"DefaultDimension"`	8	the dimension of every index for the path, when no dimensions are given
`ImageSize`, `Background`	`Automatic`, `White`	as in `Graphics`

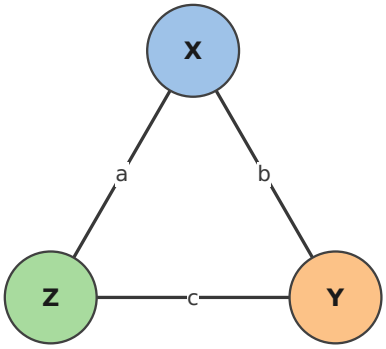
A hyperedge kept in the output:

```
EinsumDiagram["ai,bi,ci->abci", ImageSize -> 260]
```

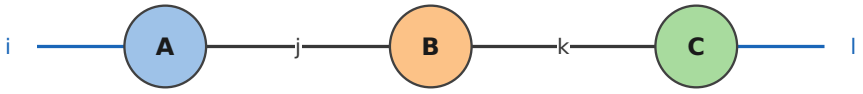


The layouts "Circle", "Line" and coordinates of your own, with names and highlighted tensors:

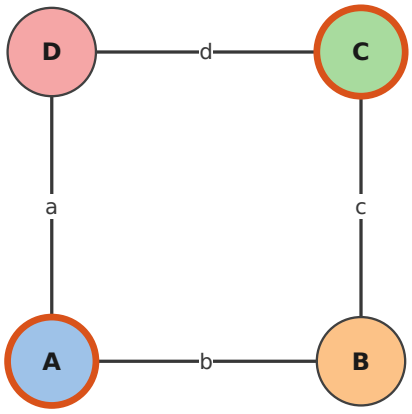
```
EinsumDiagram["ab,bc,ca->", "Layout" -> "Circle", "TensorNames" -> {"X", "Y", "Z"}]
```




```
EinsumDiagram["ij,jk,kl->il", "Layout" -> "Line"]
```



```
EinsumDiagram["ab,bc,cd,da->", "Layout" -> {{0, 0}, {1.4, 0}, {1.4, 1.4}, {0, 1.4}},
 "Highlight" -> {1, 3}]
```



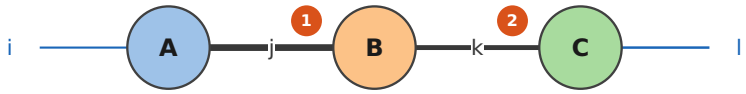
Tensors passed as symbols are named after them, and the dimensions are written:

```
SeedRandom[5]; psi = RandomReal[1, {2, 30}];
EinsumDiagram["ai,bi->ab", psi, psi]
```

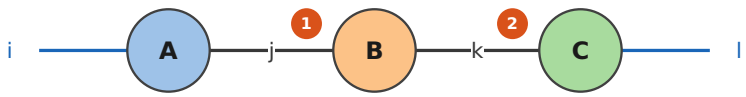


The path, with the dimensions given but not written, and with the default dimension of every index:

```
EinsumDiagram["ij,jk,kl->il", {{2, 300}, {300, 40}, {40, 3}}, "ShowPath" -> True,
 "ShowDimensions" -> False]
```




```
EinsumDiagram["ij,jk,kl->il", "ShowPath" -> True, "DefaultDimension" -> 2]
```



`"Steps" -> True` gives a list of pictures with the same plot range, to be shown by `ListAnimate`:

```
frames = EinsumDiagram["ij,jk,kl->il", {{2, 30}, {30, 40}, {40, 3}},
   "Steps" -> True];
{Length[frames], Head /@ frames}
```

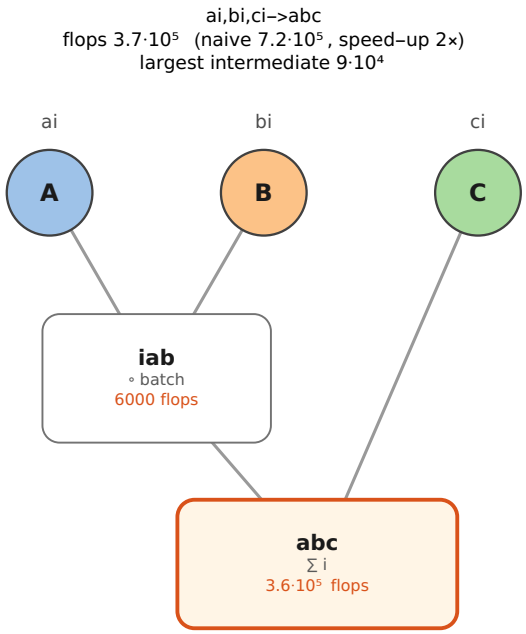
```
{3, {Graphics, Graphics, Graphics}}
```

7 EinsumTree

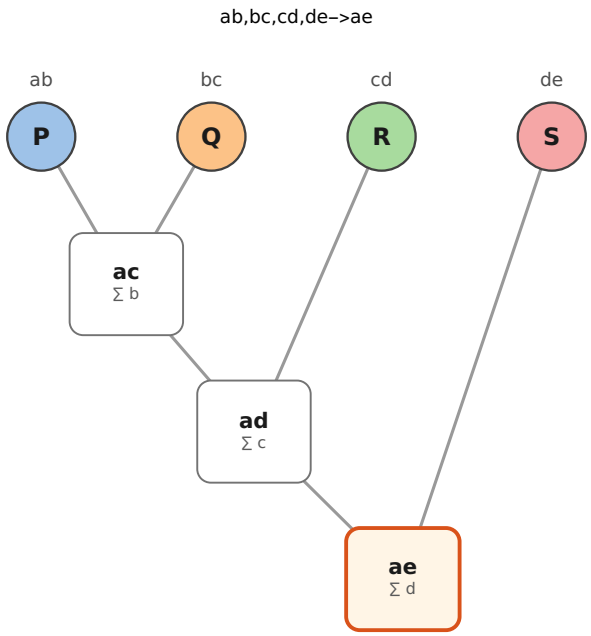
`EinsumTree`

`EinsumTree[spec, t1, t2, ...]` or `EinsumTree[spec, {dims1, dims2, ...}]` draws the contraction tree of the path: the leaves are the tensors, with their subscripts; each box is an intermediate, with its indices, the indices summed at that step (or "batch" and "outer" when none is summed) and its cost; the box of the result has an orange border. The title gives the specification, the cost of the path, the naive cost, the speed-up and the largest intermediate. Without dimensions (`EinsumTree[spec]`) the path is the one for `"DefaultDimension"` and the costs are not written. Options `"TensorNames"`, `"Path"`, `"DefaultDimension"`, `ImageSize` and `Background`, as in `EinsumDiagram`.

```
EinsumTree["ai,bi,ci->abc", {{30, 4}, {50, 4}, {60, 4}}, ImageSize -> 320]
```



```
EinsumTree["ab,bc,cd,de->ae", "TensorNames" -> {"P", "Q", "R", "S"},
 ImageSize -> 360]
```



8 The plan cache

`EinsumClearCache`

`Einsum` keeps the plan of each call, keyed by the specification, the shapes and the options `"Path"` and `"BatchMethod"`; a second call with the same key goes straight to the kernels. The cache holds up to 2000 plans and is emptied when full; `EinsumClearCache[]` empties it.

```
EinsumClearCache[]
```

9 Messages

Einsum, EinsumPath, EinsumFunction, EinsumDiagram and EinsumTree give a message and $Failed for an invalid input; Einsum::optbig and the message of an unknown option are warnings, and the computation goes on.

message	when
Einsum::spec	the specification cannot be read
Einsum::nargs	the number of tensors is not that of the subscripts
Einsum::rank	a tensor has not as many axes as its subscript has indices
Einsum::ellrank	a tensor has fewer axes than the indices named next to its ellipsis
Einsum::ell	a subscript has more than one ellipsis
Einsum::ellout	the operands have ellipsis axes, and the explicit output has no ellipsis
Einsum::dim	an index has different dimensions (other than 1)
Einsum::outrep	an index repeats in the output
Einsum::outnf	an index of the output is in no operand
Einsum::path	the path is not one of the strategies or a complete list of pairs
Einsum::array	an operand is a list that is not a rectangular array
Einsum::mismatch	an EinsumFunction gets tensors of other dimensions (section 5)
Einsum::optbig	"Optimal" is asked for more than 14 tensors: "Greedy" is used
Einsum::optx	an unknown option (section 3)
SummaUpdateCheck::latest	the latest release cannot be read from GitHub

```
Einsum["ij,jk->>ik", m, n]
```

```
Einsum::spec: Invalid einsum specification ij,jk->>ik.
$Failed
```

```
Einsum["ij,jk->ik", m]
```

```
Einsum::nargs: The specification has 2 operand(s) but 1 tensor(s) were given.
$Failed
```

```
Einsum["ijk,jk->ik", m, n]
```

```
Einsum::rank: Operand 1 has rank 2 but its subscript has 3 indices.
$Failed
```

```
Einsum["...ijk->k", m]
```

```
Einsum::ellrank: Operand 1 has rank 2, smaller than the 3 explicit indices of its
    subscript.
$Failed
```

```
Einsum["...i...->i", m]
```

```
Einsum::ell: The ellipsis can appear at most once in each subscript.
$Failed
```

```
Einsum["...i->i", m]
```

```
Einsum::ellout: The inputs have broadcast (ellipsis) dimensions, so the output
    subscript must contain "...".
$Failed
```

```
Einsum["ij,jk->ik", m, IdentityMatrix[3]]
```

```
Einsum::dim: Index j has incompatible dimensions {2, 3}.
$Failed
```

```
Einsum["ij->ii", m]
```

```
Einsum::outrep: Output index i appears more than once.
$Failed
```

```
Einsum["ij->iz", m]
```

```
Einsum::outnf: Output index z does not appear in any input.
$Failed
```

```
Einsum["ij,jk,kl->il", m, n, m, "Path" -> {{1, 2}}]
```

```
Einsum::path: Invalid contraction path {{1, 2}}. Use Automatic, "Optimal",
    "Greedy", "Sequential" or a complete list of position pairs {{i, j}, ...}.
$Failed
```

```
Einsum["ij,jk->ik", {{1, 2}, {3}}, n]
```

```
Einsum::array: Operand 1 is not a rectangular array.
$Failed
```

```
chain = StringRiffle[Table[FromCharacterCode[{96 + k, 97 + k}], {k, 15}], ","] <>
   "->ap";
EinsumPath[chain, ConstantArray[{2, 2}, 15], "Path" -> "Optimal"]["Method"]
```

```
   Einsum::optbig: The exact path search is exponential and 15 tensors exceed the
       limit of 14; using "Greedy" instead.
   Greedy
```

10 Performance

- **Packed arrays.** `Developer`ToPackedArray`, `RandomReal` and `N` give packed arrays of machine numbers, on which every kernel is fast; `Einsum` packs its operands when it can, and its results on machine numbers come out packed:

```
Developer`PackedArrayQ /@
 {Einsum["ij,jk->ik", RandomReal[1, {3, 3}], RandomReal[1, {3, 3}]],
  Einsum["i,j->ij", Range[3], Range[4]]}
```

```
   {True, True}
```

- **Loops.** In a loop, use `EinsumFunction`, or call `Einsum` always with the same shapes, so that the plan comes from the cache.
- **Large networks.** Beyond 8 tensors the default path is `"Greedy"`. When the planning time is negligible against the contraction, try `"Path" -> "Optimal"` (up to 14 tensors), and compare the two with `EinsumPath`.
- **Batched contractions.** The cost model of `"BatchMethod"` uses estimates, not timings of your machine; a kernel can be forced with `"BatchMethod" -> "Dot"` or `"Vectorized"`.
- **Sparse arrays.** A `SparseArray` stays sparse in the steps that are matrix products (`Dot`); the other kernels make it dense.
- **Timings.** `Examples/Benchmarks.wl` measures `Einsum` and `EinsumFunction` on your machine against `Dot`, `MapThread`, `TensorContract` and `Total`.

11 Limitations

- An index cannot repeat in the output (`"i->ii"` is not allowed, as in numpy).
- The exact search of the path is exponential in the number of tensors: the default is `"Greedy"` beyond 8 tensors, and `"Optimal"` is accepted up to 14.
- A `SparseArray` stays sparse only in the steps that are matrix products.
- The constants of the cost model of the batched contractions are estimates, not calibrated on your machine.

12 Citing Summa

If you use Summa in published work, please cite it as described in `CITATION.cff`:

```
@software{Summa,
  author  = {Africani, Nazzareno},
  title   = {Summa: fast Einstein summation (einsum) for the Wolfram Language},
  version = {1.0.0},
  year    = {2026},
  url     = {https://github.com/n4sd4q/Summa}
}
```

13 License

Summa is free software, released under the GNU General Public License, version 3 or later (`LICENSE`), with two additional terms under its section 7 (`NOTICE`): the attribution "Summa, by Nazzareno Africani" and the contents of `NOTICE` must be kept in all copies and modified versions, in their documentation and in the message printed when the program is loaded in a notebook; and a modified version must be marked as different from the original: it must not be distributed under the name "Summa" alone, and its documentation and its loading message must say that it is a modified version of Summa by Nazzareno Africani, by whom and when it was modified. Copyright © 2026 Nazzareno Africani (SISSA, Trieste).